

Optimization for Data Science

Lecture 3: Fundamentals of Unconstrained Optimization

Prof. Udit Bhatia

Indian Institute of Technology Gandhinagar

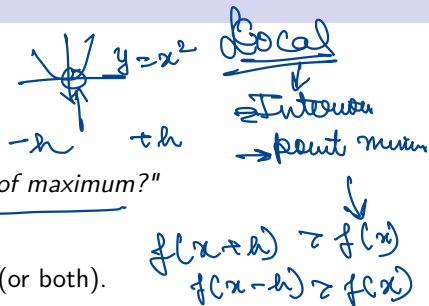
bhatia.u@iitgn.ac.in

January 24, 2025

Outline

- 1 Refresher: Critical Points and Unbounded Functions
- 2 Unconstrained Optimization: Setting
- 3 Gradient Descent (Steepest Descent)
- 4 Newton's Method
- 5 Example Comparisons
- 6 Links to Data Science Problems
- 7 Line Search and Damping
- 8 Summary & Next Steps

Local Extrema vs. Unboundedness



Key Question from Last Class:

"Does an unbounded function also have gradient = 0 at its point of maximum?"

Clarifications:

- A function can be **unbounded above** or **unbounded below** (or both).
- **Local extrema (min or max)** can still exist even if the function is unbounded globally.
- For a **local** minimum or maximum in a differentiable function, we generally require $\nabla f(\mathbf{w}^*) = \mathbf{0}$. This is a *necessary condition* for an **interior** critical point.
- *Global* maximum might not exist if the function is unbounded above.

Example: Unbounded Functions

Example 1: $f(w) = w^2$

- $\min_{w \in \mathbb{R}} f(w) = 0$ (achieved at $w = 0$).
- The function is **unbounded above**. As $w \rightarrow \pm\infty$, $f(w) \rightarrow +\infty$.
- $\nabla f(w) = 2w$. The only critical point is at $w = 0$, where the gradient is zero.
- This critical point is a *global* (and local) **minimum**.

Example 2: $g(w) = -w^2$

- $\max_{w \in \mathbb{R}} g(w) = 0$ (achieved at $w = 0$).
- The function is **unbounded below**. As $w \rightarrow \pm\infty$, $g(w) \rightarrow -\infty$.
- $\nabla g(w) = -2w$. The only critical point is at $w = 0$, where the gradient is zero.
- This critical point is a *global* (and local) **maximum**.

What if the Function Has No Maximum?

Consider: $h(w) = e^w$

$$f(x) = e^x$$

- $h(w)$ is **unbounded above**. There's no finite w at which $h(w)$ reaches a global maximum.
- The derivative is $h'(w) = e^w$, which is never zero for any real w .
- Hence, there is **no critical point** in $\mathbb{R}$.
- Consequently, no local maximum or local minimum exists in $\mathbb{R}$.

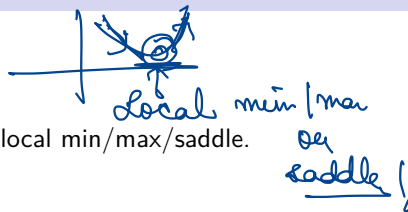
Takeaway:

- A function can fail to have a global maximum if it is unbounded above.
- The gradient condition $\nabla f(\mathbf{w}) = \mathbf{0}$ applies only to points that are local extrema (or saddle points).
- If no such local extrema exist, then you simply do **not** find any point with zero gradient.

Why is $\nabla f = 0$ a Necessary Condition?

Local Optimality Condition (smooth f):

$\nabla f(\mathbf{w}^*) = \mathbf{0}$ is a necessary condition for a local min/max/saddle.



Interpretation:

- If the gradient is $\neq 0$ at $\mathbf{w}^*$, you can still "move" in some direction to change f . Hence, it cannot be a local optimum.
- However, $\nabla f = 0$ does *not* guarantee a *global* optimum or even a *local* optimum—it could be a **saddle point**.
- For *bounded* and *continuous* functions on a closed domain, a global optimum must exist, but the function might achieve it on the boundary (where other conditions apply).
- For *unbounded* functions, a global maximum (or minimum) may not exist at all. In that case, there may be no critical point that corresponds to a global extremum.

Unconstrained Optimization: Overview

General Problem:

$$\min_{\mathbf{w} \in \mathbb{R}^d} f(\mathbf{w}),$$

where $\mathbf{w}$ is free to vary in $\mathbb{R}^d$ (no explicit constraints).

$$\vec{w} = C_1 x_1 + C_2 x_2 + \dots + C_d x_d$$

$$\rightarrow \mathbb{R}^d$$

$$x_d \rightarrow (-\infty, \infty)$$

Examples in Data Science:

- ✓ *Least Squares Regression*: no direct bounds on coefficients. $\rightarrow (-\infty, \infty)$
- ✓ *Logistic Regression*: parameters can be any real value. $\in [0, 1]$
- *Neural Networks*: typically unconstrained weights (unless we add weight-penalties).

} optimization problem.

Optimality Conditions (smooth f):

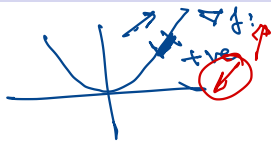
$$\checkmark \nabla f(\mathbf{w}^*) = \mathbf{0}, \quad \checkmark \nabla^2 f(\mathbf{w}^*) \succeq 0.$$

If f is convex, $\mathbf{w}^*$ is a global minimizer.

↑
necessary

↑
Hessian

Gradient Descent: Big Picture



Lasso, Ridge & Elastic net
 $= L1 ; L2 \dots$

$$\nabla f$$

$$x \frac{\partial f}{\partial x} x$$

$$[x]^2$$

Key Idea:

- We use $\nabla f(\mathbf{w})$ (the gradient) to find the direction of steepest ascent.
- We move **opposite** to that direction (steepest descent).

Update Rule (in vector form):

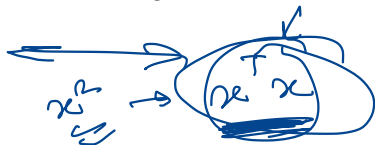
sensitivity of obj
 for w.r.t. α

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \alpha \nabla f(\mathbf{w}^{(k)})$$

magnitude α direction part $\nabla f(\mathbf{w}^{(k)})$

squaring the matrix $\|\cdot\|_2$

where $\alpha > 0$ is the step size or learning rate.



$$w \left(z_1^2 + z_2^2 \right)$$

$$y(a, x_0) = y = ax + b$$

$$\nabla y = x \uparrow \rightarrow 0$$

Gradient Descent: 1D Derivation (Step by Step)

Consider a **1D function** $f(w)$. We want to find $\min_{w \in \mathbb{R}} f(w)$.

Idea:

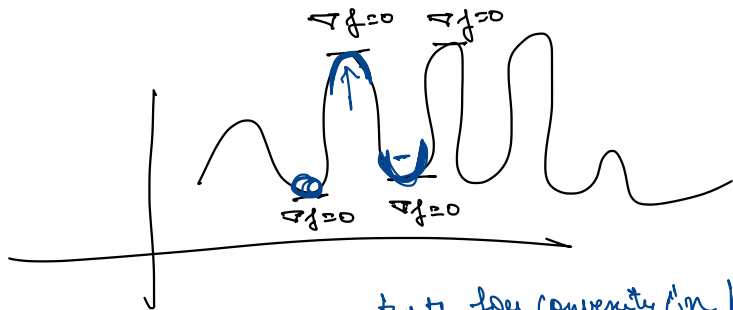
- 1 Look at the derivative $f'(w)$.
- 2 If $f'(w) > 0$, the function is increasing at w .
- 3 If $f'(w) < 0$, the function is decreasing at w .
- 4 To *decrease* f , we move in the direction $-f'(w)$.

∇ : gradient!
 ∇^2 $\nabla \times$
 ∇^n sliding!

Update in 1D:

$$w^{(k+1)} = w^{(k)} - \alpha f'(w^{(k)}).$$

Interpretation: - Subtracting a small fraction α of the derivative. - If $f'(w^{(k)})$ is large and positive, we move left quickly. - If it's large and negative, we move right quickly.



tests for convexity on b !

simulated annealing ✓

Gradient Descent: Multivariate Step by Step

Now let $\mathbf{w} \in \mathbb{R}^d$. The gradient is $\nabla f(\mathbf{w}) \in \mathbb{R}^d$.

Geometric meaning of ∇f :

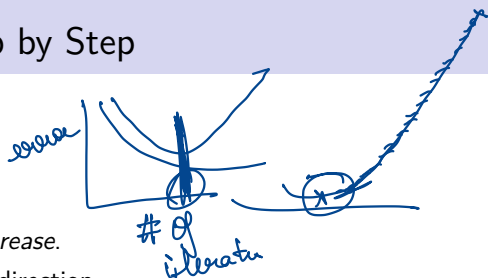
- $\nabla f(\mathbf{w})$ points to the direction of steepest increase.
- Its magnitude is how fast f increases in that direction.

Update formula:

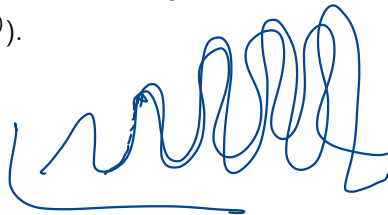
$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \alpha \nabla f(\mathbf{w}^{(k)}).$$

Algorithmically:

- 1 Compute gradient $\mathbf{g}^{(k)} = \nabla f(\mathbf{w}^{(k)})$.
- 2 Update: $\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \alpha \mathbf{g}^{(k)}$.
- 3 Repeat until convergence.



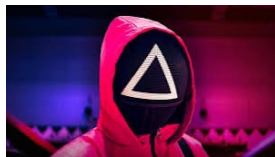
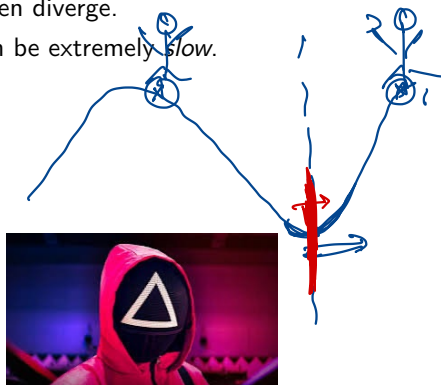
$$\nabla f \approx 0$$



Choosing the Step Size α

"heuristically"

- If α is too **large**, the method may overshoot and even diverge.
- If α is too **small**, convergence is guaranteed but can be extremely *slow*.
- Often we use:
 - A *fixed* small α .
 - A decaying α schedule (in machine learning).
 - A **line search** method to choose α at each step.



Motivation for Newton's Method

Shortcoming of Gradient Descent:

- Uses only the *first derivative* (gradient) to decide direction.
- If the function is very curved, convergence might be slow (zig-zag in narrow valleys).

Newton's Idea:

- Use the **second derivative** (Hessian) to get a local quadratic approximation.
- $\Rightarrow$ Move more directly toward the minimizer of that local quadratic model.

Newton's Update (in short):

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - [\nabla^2 f(\mathbf{w}^{(k)})]^{-1} \nabla f(\mathbf{w}^{(k)}).$$

Calculation

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \nabla^2 f(\mathbf{w}^{(k)})^{-1} \nabla f(\mathbf{w}^{(k)})$$

Newton's Method: 1D Derivation (Step by Step)

Setup: $f(w) \in \mathbb{R}$. Let $w^{(k)}$ be current estimate.

Step 1. Taylor Expansion (2nd order) around $w^{(k)}$:

$$f(w) \approx \underbrace{f(w^{(k)})}_{\text{red line}} + \underbrace{f'(w^{(k)})}_{\text{red line}} \underbrace{(w - w^{(k)})}_{\text{red line}} + \frac{1}{2} \underbrace{f''(w^{(k)})}_{\text{red line}} \underbrace{(w - w^{(k)})^2}_{\text{red line}}.$$

Handwritten notes:
 $w - w^R = h$
 $g(x+h) = g(x) + h g'(x)$
 $x+h = w$
 $h = w - x$

Step 2. Minimize this approximate quadratic in w :

Handwritten: $f(w) \Rightarrow$

$$\frac{d}{dw} \left[f(w^{(k)}) + f'(w^{(k)})(w - w^{(k)}) + \frac{1}{2} f''(w^{(k)})(w - w^{(k)})^2 \right] = 0.$$

Handwritten: $\frac{\partial f(w)}{\partial w} = 0$

Newton's Method: 1D Derivation (cont.)

Derivative of the approximation:

$$\frac{d}{dw} [f(w^{(k)})] = 0 \quad (\text{constant in } w) \Leftrightarrow$$

$$\frac{d}{dw} [f'(w^{(k)})(w - w^{(k)})] = \underline{f'(w^{(k)})},$$

$$\frac{d}{dw} \left[\frac{1}{2} f''(w^{(k)})(w - w^{(k)})^2 \right] = \underline{f''(w^{(k)})(w - w^{(k)})}.$$

d-dimension
↓
"d x d"

Setting sum to zero:

$$f'(w^{(k)}) + f''(w^{(k)})(w - w^{(k)}) = 0.$$

Hence

$$w - w^{(k)} = -\frac{f'(w^{(k)})}{f''(w^{(k)})}.$$

So

$$\boxed{w^{(k+1)} = w^{(k)} - \frac{f'(w^{(k)})}{\underline{f''(w^{(k)})}}. \quad \#}$$

Hessian⁻¹
↓
 $f'' = \nabla^2 f$

Newton's Method: Multivariate Derivation (Step by Step)

Now let $\mathbf{w} \in \mathbb{R}^d$. Define $\nabla f(\mathbf{w}) \in \mathbb{R}^d$ and $\nabla^2 f(\mathbf{w}) \in \mathbb{R}^{d \times d}$.

Second-order Taylor expansion around $\mathbf{w}^{(k)}$:

$$f(\mathbf{w}) \approx f(\mathbf{w}^{(k)}) + \nabla f(\mathbf{w}^{(k)})^\top (\mathbf{w} - \mathbf{w}^{(k)}) + \frac{1}{2}(\mathbf{w} - \mathbf{w}^{(k)})^\top \nabla^2 f(\mathbf{w}^{(k)}) (\mathbf{w} - \mathbf{w}^{(k)}).$$

Newton's Method: Multivariate Derivation (cont.)

We want $\min_{\mathbf{w}}$ (that quadratic approximation).

Compute gradient w.r.t. $\mathbf{w}$ and set $= \mathbf{0}$:

$$\nabla f(\mathbf{w}^{(k)}) + \nabla^2 f(\mathbf{w}^{(k)}) (\mathbf{w} - \mathbf{w}^{(k)}) = \mathbf{0}.$$

Solve for $\mathbf{w} - \mathbf{w}^{(k)}$:

$$\nabla^2 f(\mathbf{w}^{(k)}) (\mathbf{w} - \mathbf{w}^{(k)}) = -\nabla f(\mathbf{w}^{(k)}).$$

$$\mathbf{w} - \mathbf{w}^{(k)} = -[\nabla^2 f(\mathbf{w}^{(k)})]^{-1} \nabla f(\mathbf{w}^{(k)}).$$

Newton's Method: Final Update (Vector Form)

"newton" solved!

$$\longleftrightarrow \boxed{\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - [\nabla^2 f(\mathbf{w}^{(k)})]^{-1} \nabla f(\mathbf{w}^{(k)})} \longleftrightarrow$$

Interpretation:

- $[\nabla^2 f(\mathbf{w}^{(k)})]^{-1}$ is a local curvature adjustment.
- If Hessian is large in some direction, the step is smaller.
- Converges **quadratically** near the optimum if the Hessian is positive definite.

Newton's Method: Practical Points

- **Hessian Computation:** For large d , computing (or storing) $\nabla^2 f(\mathbf{w})$ is expensive. $\Rightarrow$ We often use *Quasi-Newton* methods (e.g., BFGS, L-BFGS).
- **Hessian Inversion:** - Actually, we solve $\nabla^2 f \mathbf{p} = -\nabla f$ instead of forming the inverse directly.
- **Line Search or Damping:** - The full Newton step can be too aggressive if far from the optimum.
- We might scale step size with $\alpha_k \leq 1$.
- **Local Minima vs. Global:** - Newton's method is local. If f is non-convex, it may get stuck in a local minimum or saddle.

Comparison: 1D Quadratic $f(w) = w^2$

Gradient Descent:

$$w^{(k+1)} = w^{(k)} - \alpha (2 w^{(k)}).$$

Requires multiple steps to reach 0, depending on α .

Newton's Method:

$$w^{(k+1)} = w^{(k)} - \frac{2 w^{(k)}}{2} = 0 \quad (\text{in one step}).$$

Insight: For a perfect quadratic, Newton's method can reach the solution in **one iteration**.

2D Example: $f(w_1, w_2) = w_1^2 + 2 w_2^2$

$$\nabla f = \begin{pmatrix} 2w_1 \\ 4w_2 \end{pmatrix}, \quad \nabla^2 f = \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}.$$

Newton's Method:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \begin{pmatrix} 2 & 0 \\ 0 & 4 \end{pmatrix}^{-1} \begin{pmatrix} 2 w_1^{(k)} \\ 4 w_2^{(k)} \end{pmatrix} = \mathbf{0}.$$

Conclusion: One step to $(0, 0)$.

Gradient Descent: would take many steps if α is small (though stable).

Least Squares: A Perfect Quadratic Case

Objective:

$$f(\mathbf{w}) = \|X\mathbf{w} - \mathbf{y}\|^2 = (\mathbf{w}^\top X^\top - \mathbf{y}^\top)(X\mathbf{w} - \mathbf{y}).$$

Gradient:

$$\nabla f(\mathbf{w}) = 2X^\top(X\mathbf{w} - \mathbf{y}) = 2(X^\top X\mathbf{w} - X^\top \mathbf{y}).$$

Hessian:

$$\nabla^2 f(\mathbf{w}) = 2X^\top X \quad (\text{constant in } \mathbf{w}).$$

Least Squares: Newton's Method

Newton's Update:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - (2X^\top X)^{-1} \nabla f(\mathbf{w}^{(k)}).$$

Since $\nabla f(\mathbf{w}) = 2(X^\top X \mathbf{w} - X^\top \mathbf{y})$,

$$\begin{aligned}\mathbf{w}^{(k+1)} &= \mathbf{w}^{(k)} - (2X^\top X)^{-1} \cdot 2(X^\top X \mathbf{w}^{(k)} - X^\top \mathbf{y}). \\ &= \mathbf{w}^{(k)} - (X^\top X)^{-1} (X^\top X \mathbf{w}^{(k)} - X^\top \mathbf{y}). \\ &= (X^\top X)^{-1} X^\top \mathbf{y}.\end{aligned}$$

One step! (assuming $X^\top X$ invertible).

Logistic Regression: Convex but Not Quadratic

Objective:

$$\min_{\mathbf{w}} \sum_{i=1}^n \left[-y_i \ln(\sigma(\mathbf{w}^\top \mathbf{x}_i)) - (1 - y_i) \ln(1 - \sigma(\mathbf{w}^\top \mathbf{x}_i)) \right],$$

with $\sigma(z) = 1/(1 + e^{-z})$.

Notes:

- The Hessian is *not* constant, so we can't solve in one step.
- $\nabla^2 f$ is typically well-defined but must be updated each iteration.
- Newton's method converges faster than simple gradient descent, but more expensive per iteration.

Line Search or Damping

Newton's Method:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - [\nabla^2 f(\mathbf{w}^{(k)})]^{-1} \nabla f(\mathbf{w}^{(k)}).$$

In practice, we often do:

$$\mathbf{w}^{(k+1)} = \mathbf{w}^{(k)} - \alpha_k [\nabla^2 f(\mathbf{w}^{(k)})]^{-1} \nabla f(\mathbf{w}^{(k)}),$$

where $0 < \alpha_k \leq 1$.

Why?

- If $\mathbf{w}^{(k)}$ is **far** from optimum, the pure Newton step might overshoot.
- We adapt α_k via *backtracking line search* (Armijo rule) or *trust-region* methods.

Comparing Gradient Descent and Newton's Method

- **Gradient Descent:**

- Low per-iteration cost (only need gradient).
- Potentially *many* steps if the problem is ill-conditioned.
- Heavily used in high-dimensional ML (especially with mini-batches).

- **Newton's Method:**

- Faster (often quadratic) convergence near optimum.
- High per-iteration cost: must handle Hessian.
- Often used for moderate dimensions or specialized problems (e.g., some iterative solvers or quasi-Newton approximations).

Key Takeaways

Unconstrained Optimization:

- Solve $\min_{\mathbf{w}} f(\mathbf{w})$ with no explicit constraints.
- Important building block for many ML algorithms.

Gradient Descent:

- First-order method (requires ∇f).
- Step size α is crucial.

Newton's Method:

- Second-order method (uses $\nabla^2 f$).
- Potentially very fast convergence near solution.
- Costly Hessian computations for large-scale problems.

Next Steps

- **Line Search Methods:** Armijo, Wolfe, etc.
- **Quasi-Newton Methods:** BFGS, L-BFGS (avoid full Hessian).
- **Stochastic Methods:** SGD, mini-batch gradient descent.
- **Advanced Topics:** Trust-region methods, proximal methods, and constrained optimization.

Questions?

Thank You!